

Proyecto Final

Sistema de Gestión de Inventario Rural

PostgreSQL · ACID

MongoDB · BASE

Bases de Datos II — Semana 14

Santiago Chavarro H.

Juan Pablo Sánchez

Grupo 10

¿Por qué una sola BD no es suficiente?

X Solo Relacional (SQL)

- Esquema rígido
- Dificulta atributos variables por lote de café, cacao, panela
- Cuello de botella en escrituras intensivas

X Solo Documental (NoSQL)

- Sin integridad referencial
- Sin garantías transaccionales para ventas e inventario
- Riesgo de stock negativo o ventas sin respaldo

OK Solución: Persistencia Polígloa

Asignar a cada dato el motor más adecuado según su naturaleza, patrones de acceso y requisitos de consistencia.

PostgreSQL (ACID) para transacciones críticas + MongoDB (BASE/CAP) para datos flexibles y dinámicos

ACID · BASE · Teorema CAP

PG Modelo ACID — PostgreSQL

A Atomicidad

Todo o nada. La transacción se ejecuta completa o se revierte.

C Consistencia

La BD siempre pasa de un estado válido a otro.

I Aislamiento

Transacciones concurrentes como si fueran secuenciales.

D Durabilidad

Una transacción confirmada persiste ante fallos.

MDB Modelo BASE — MongoDB

Basically Available — siempre disponible

Soft-state — estado puede cambiar con el tiempo

Eventually consistent — consistencia eventual

= Teorema CAP (Brewer, 2000)

Un sistema distribuido solo puede garantizar
2 de 3 propiedades simultáneamente:



Consistencia



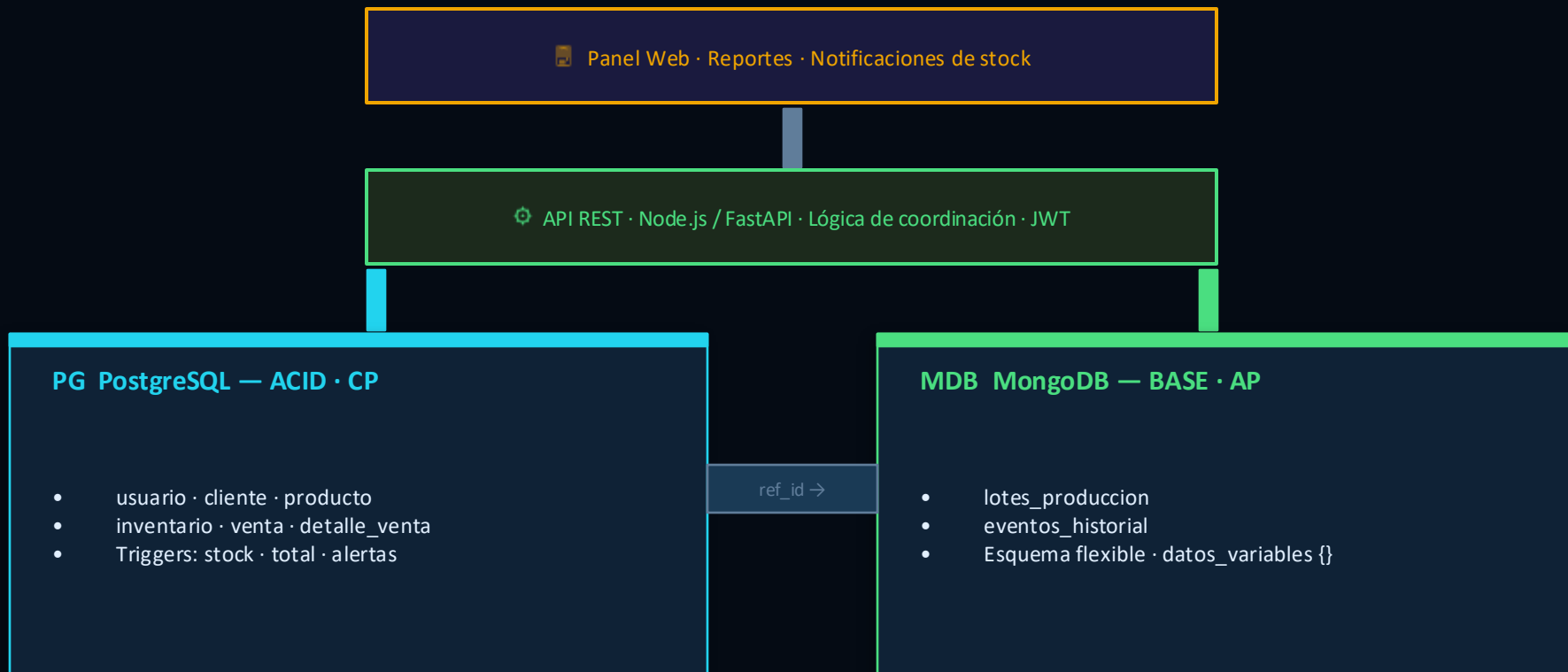
Disponibilidad



Tolerancia
particiones

MongoDB elige AP → disponibilidad + tolerancia a particiones

Diagrama de Persistencia Políglota



Integración por referencias de ID — sin duplicar datos — coordinación en capa de aplicación

¿Dónde vive cada dato y por qué?

Entidad	Motor	Modelo	CAP	Razón principal
usuario	PostgreSQL	ACID	CP	Autenticación confiable, unicidad de email
cliente	PostgreSQL	ACID	CP	FK en ventas, integridad referencial
producto	PostgreSQL	ACID	CP	Precio consistente en ventas simultáneas
inventario	PostgreSQL	ACID	CP	Descuento atómico con triggers
venta	PostgreSQL	ACID	CP	Transacción financiera crítica
detalle_venta	PostgreSQL	ACID	CP	Resolución N:M — activa trigger de stock
lotes_produccion	MongoDB	BASE	AP	Esquema flexible por tipo de producto
eventos_historial	MongoDB	BASE	AP	Log inmutable, alta frecuencia escritura
6 entidades → PostgreSQL (ACID)				2 entidades → MongoDB (BASE/CAP)

Lógica de negocio directamente en la BD

~~ trg_inventario_updated

BEFORE UPDATE

Durabilidad

Actualiza updated_at automáticamente en cada modificación de stock.

trg_recalcular_total

AFTER I/U/D

Consistencia

Recalcula total de venta = $\Sigma(\text{cantidad} \times \text{precio})$. Nunca queda desincronizado.

* trg_descontar_stock

AFTER INSERT

Atomicidad

Verifica stock disponible y descuenta. Si hay insuficiencia → ROLLBACK completo.

! trg_alerta_stock

AFTER UPDATE

Consistencia

Emite NOTICE si $\text{stock} \leq \text{mínimo}$. Capturado por la app para notificar al distribuidor.

* Atomicidad: el trigger protege el inventario

SQL Ejecutado

```
-- Intentar vender 999.999 kg de café
-- (stock real: 480 kg)

INSERT INTO detalle_venta
  (venta_id, producto_id,
   cantidad, precio_unitario)
VALUES
  (1, 1, 999999, 12500);
```

```
ERR ERROR: Stock insuficiente — pedido=999999, stock actual=480
→ Transacción revertida (ROLLBACK automático)
```

Flujo del trigger

- 1 INSERT en detalle_venta
- 2 Trigger se dispara automáticamente
- 3 Consulta stock actual (480 kg)
- 4 $999999 > 480 \rightarrow \text{RAISE EXCEPTION}$
- 5 ROLLBACK — stock sin cambios
- 6 Integridad preservada OK

Datos flexibles para el campo rural

[] lotes_produccion

Justificación BASE/AP:

Cada producto rural tiene atributos distintos.
El café necesita variedad y altitud; el cacao,
días de fermentación; la miel, valor Brix.
Esquema rígido requeriría decenas de columnas.

Ejemplo datos_variables:

```
// Café:
{ variedad: "Castillo",
  altitud_msnm: 1750 }

// Cacao:
{ dias_fermentacion: 6,
  codigo_trazabilidad:
    "CACAO-MACIZO-2025" }
```

log eventos_historial

Log inmutable de auditoría del sistema:

[] produccion

sale venta

fix ajuste_inventario

ret devolucion

! alerta_stock

Justificación BASE/AP:

Alta frecuencia de escritura. Consistencia eventual aceptable — si un evento tarda ms en replicarse no afecta la venta ya confirmada en PG.

Costos asumibles vs. beneficios obtenidos

Aspecto	PostgreSQL	MongoDB
Consistencia	Fuerte — inmediata	Eventual — milisegundos
Esquema	Rígido — requiere migraciones	Flexible — sin migraciones
Transacciones	Nativas distribuidas	Lógica en capa de aplicación
Escalabilidad	Vertical (más RAM/CPU)	Horizontal (más nodos)
Caso de uso	Ventas · inventario · facturas	Lotes · auditoría · métricas



Complejidad operativa:

Dos motores = dos sistemas de backup, autenticación y monitoreo.



Flexibilidad de esquema:

datos_variables absorbe cualquier atributo de lote sin migraciones.



Consistencia entre motores:

No existe transacción distribuida PG ↔ MongoDB. La compensación está en la app.



Rendimiento por dominio:

Cada motor brilla en su caso de uso. El sistema es más escalable que una solución monolítica.

Del prototipo a producción

sec Seguridad y autenticación

JWT + bcrypt + sistema de roles (administrador, vendedor, supervisor).

web API REST

Node.js/Express o FastAPI coordinando ambas capas con paginación y manejo de errores.

msg Consistencia entre motores

Patrón Outbox o cola de mensajes (Kafka/RabbitMQ) para eventos exactamente una vez.

cloud Despliegue Cloud Native

PostgreSQL en Azure/AWS · MongoDB Atlas · contenedores Docker Compose.

chart Panel web en tiempo real

Gráficas de ventas e inventario, alertas de stock por correo, exportación PDF/Excel.

test Pruebas y CI/CD

Tests de integración para flujos críticos + pipeline automatizado de despliegue.

Conclusiones

01

La selección deliberada del motor según la naturaleza de cada dato produce sistemas más robustos que una solución monolítica.

02

Los triggers de PostgreSQL encapsulan la lógica crítica directamente en la BD, garantizando ACID independientemente del cliente.

03

MongoDB absorbe la variabilidad de lotes de producción (café, cacao, panela, miel) sin imponer un esquema rígido.

04

La arquitectura polígota es escalable: la mejora de mayor impacto es el patrón Outbox para consistencia entre motores.